Relazione del progetto UniManager

Gestione degli esami di un ateneo

Programmazione ad Oggetti

kuetchetadomdjouTheophile .

Abstract

UniManager è un'applicazione desktop in Java per gestire gli esami di un ateneo: studenti, corsi, appelli d'esame, iscrizioni, voti e medie. Il progetto adotta l'architettura Model–View–Controller con interfaccia grafica Swing. Le regole del dominio (matricola unica, voto tra 18 e 30, lode solo con 30, un solo voto per studente e appello, voto solo se iscritti

)ilcodice è nelrepositoryhttps:
//github. com/kuetchetheophile98- cloud/OOP24-unimanager.

Indice

1 Analisi **2**
- 1.1 Descrizione e requisiti . . . 2
- 1.2 Modello del dominio . . . 3

2 Design **4**
- 2.1 Architettura . . . 4
- 2.2 Design dettagliato . . . 4
  - 2.2.1 Aggiornare la vista quando il modello cambia . . . 4
  - 2.2.2 Calcolare medie diverse . . . 5
  - 2.2.3 Garantire le regole del dominio . . . 6
  - 2.2.4 Proteggere lo stato interno . . . 6

3 Sviluppo **7**
- 3.1 Testing automatizzato . . . 7
- 3.2 Note di sviluppo . . . 7

4 Commenti finali **9**
- 4.1 Autovalutazione e lavori futuri . . . 9

A Guida utente **10**
- A.1 Avvio . . . 10
- A.2 Uso . . . 10

B) Esercitazioni di laboratorio **13**

Capitolo 1

Analisi

1.1 Descrizione e requisiti

UniManager permette a un operatore di segreteria di tenere traccia della carriera d'esame degli studenti: inserisce studenti e corsi, pubblica gli appelli di ciascun corso, iscrive gli studenti agli appelli, registra i voti ottenuti e consulta le medie.

Requisiti funzionali

- Inserire uno studente con nome, cognome e matricola; la matricola è unica nell'ateneo.
- Inserire un corso con nome e crediti formativi (CFU), da 1 a 30.
- Creare un appello per un corso esistente, indicando data e luogo.
- Iscrivere uno studente a un appello; la stessa iscrizione non può essere ripetuta.
- Registrare il voto di uno studente a un appello, con queste regole: il voto è compreso tra 18 e 30, la lode è ammessa solo con 30, lo studente deve essere iscritto all'appello e non può avere due voti per lo stesso appello.
- Calcolare, per uno studente, la media aritmetica e la media ponderata sui CFU dei suoi voti; calcolare, per un corso, la media aritmetica dei voti ottenuti in tutti i suoi appelli. La lode vale 30.
- Consultare in tabella l'elenco di studenti, corsi, appelli e voti.

Requisiti non funzionali

- Una violazione delle regole non deve far terminare l'applicazione: l'utente riceve un messaggio chiaro e può correggere l'input.
- La logica del dominio deve poter essere provata senza avviare la grafica.
- L'interfaccia deve riflettere subito ogni modifica dei dati, senza azioni di aggiornamento

- Ilprogettodevecompilareedeseguirsicone JDK

Fuori ambito

I dati risiedono in memoria e non vengono salvati alla chiusura. Non sono previste modifica o cancellazione di elementi già inseriti, né autenticazione degli operatori.

1.2 Modello del dominio

Il dominio ruota attorno a quattro entità. Uno *studente* è identificato da nome, cognome e matricola. Un *corso* ha un nome e un peso in CFU. Un *appello* è una sessione d'esame di un corso, in una certa data e luogo, a cui si iscrivono degli studenti. Un *voto* è l'esito di uno studente a un appello. Le relazioni sono mostrate in Figura 1.1.

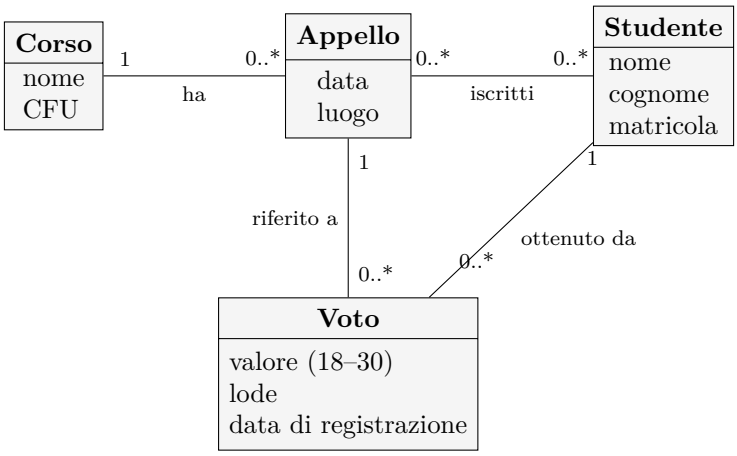


Figura 1.1: Modello del dominio.

Il dominio impone queste regole, che il modello dovrà garantire: la matricola identifica un solo studente; i CFU sono tra 1 e 30; un voto esiste solo se lo studente è iscritto all'appello; per ognicoppiastudente -appelloc'èalpiù unvoto.

Capitolo 2

Design

2.1 Architettura

L'applicazione segue il pattern Model–View–Controller (Figura 2.1), con un package per ciascuna parte.

- **Model** (`it.unibo.unimanager.model`): le entità `Studente`, `Corso`, `Appello`, `Voto` e la classe `Ateneo`, che custodisce le collezioni e fa rispettare le regole del dominio. Non conosce né il controller né la vista.
- **Controller** (`it.unibo.unimanager.controller`): la classe `UniManagerController` riceve dalla vista i dati grezzi (per esempio la data come testo), li traduce in chiamate al modello e restituisce i risultati. Non dipende da Swing, perciò si può provare senza grafica.
- **View** (`it.unibo.unimanager.view`): `MainFrame` è una finestra con quattro schede, ciascuna un pannello Swing. La vista non modifica mai il modello direttamente: ogni azione passa dal controller.

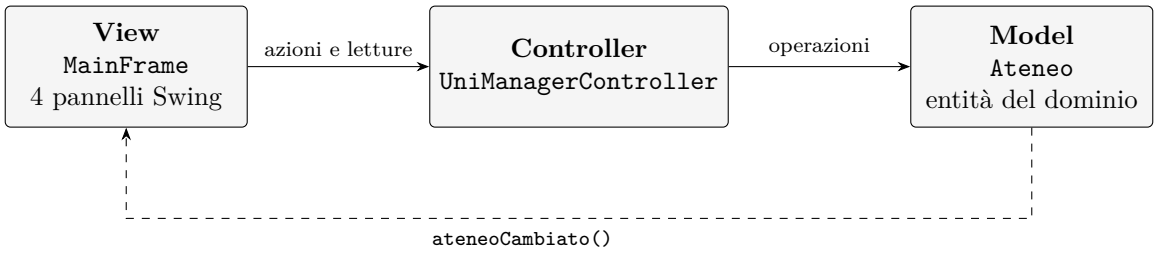


Figura 2.1: Architettura MVC. La freccia tratteggiata è la notifica del modello alla vista.

Il flusso è sempre lo stesso. L'utente agisce su un pannello; il pannello chiama il controller; il controller chiama il modello. Se l'operazione riesce, il modello avvisa gli osservatori registrati e la vista rilegge i dati dal controller e si aggiorna. Se il modello solleva un'eccezione di dominio, il pannello la cattura e mostra il messaggio in una finestra d'errore; i dati restano invariati.

2.2 Design dettagliato

2.2.1 Aggiornarelavistaquandoilmodellocambia

Problema. Dopo ogni inserimento le tabelle e le liste a discesa di tutte le schede devono cambiare. Il modello però non deve conoscere Swing.

Soluzione. Pattern *Observer*. L'interfaccia funzionale `AteneoListener` ha un solo metodo, `ateneoCambiato()`. `Ateneo` tiene una lista di osservatori (`aggiungiOsservatore`) e li avvisa alla fine di ogni modifica riuscita (`notifica`). All'avvio, `MainFrame` registra un osservatore che chiede a tutti i pannelli di aggiornarsi (`MainFrame.java, righe 33-34`). I pannelli implementano la piccola interfaccia `Aggiornabile`, così la finestra li tratta in modo uniforme.

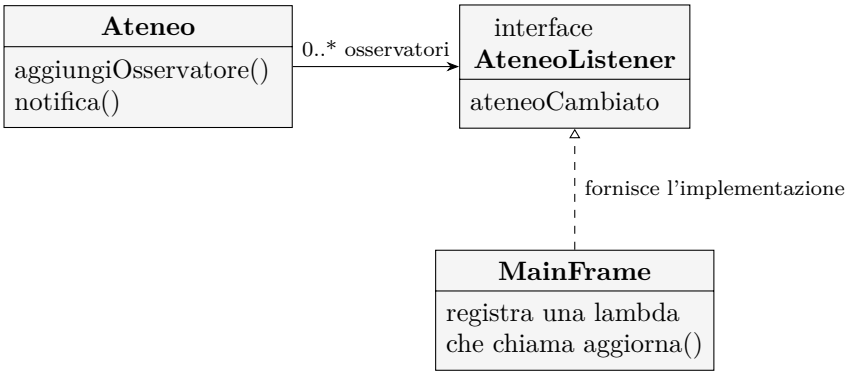


Figura 2.2: Pattern Observer per l'aggiornamento della vista.

Alternative scartate. Far aggiornare ogni pannello direttamente dal controller avrebbe legato il controller ai dettagli della vista; far rileggere le tabelle a intervalli regolari avrebbe sprecato lavoro e ritardato la risposta.

2.2.2 Calcolare medie diverse

Problema. Lo studente ha due medie sensate, aritmetica e ponderata sui CFU, e se ne potrebbero aggiungere altre. `Ateneo` non deve cambiare a ogni nuova media.

Soluzione. Pattern *Strategy*. L'interfaccia `CalcoloMedia` dichiara `calcola(voti)`. `MediaAritmetica` e `MediaPonderata` la realizzano. `Ateneo.mediaStudente` riceve la strategia come parametro e la applica ai voti dello studente (`Ateneo.java, righe 151--154`). Un caso senza voti produce un `OptionalDouble` vuoto, non un valore inventato.

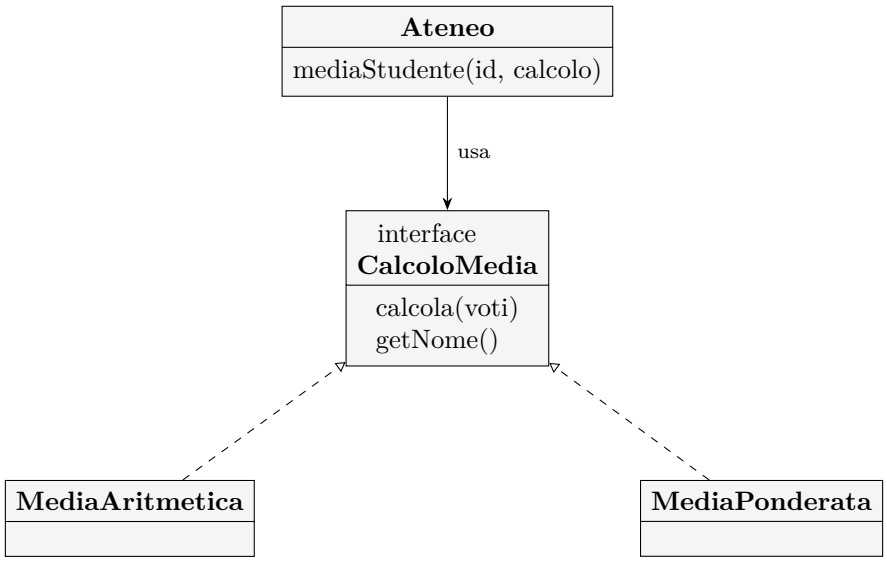


Figura 2.3: Pattern Strategy per il calcolo delle medie.

Alternativa scartata. Un metodo con un `if` sul tipo di media costringerebbe a modificare `Ateneo` per ogni nuova media.

2.2.3 Garantire le regole del dominio

Problema. Le regole devono valere qualunque sia il chiamante. Se fossero controllate solo nella grafica, un altro chiamante, per esempio un test, potrebbe aggirarle.

Soluzione. Le entità sono immutabili e verificano i propri vincoli nel costruttore: per esempio `Voto` rifiuta valori fuori da 18–30 e una lode senza 30 (`Voto.java, righe 35--41`). Le regole che coinvolgono più entità stanno in `Ateneo`: la verifica dell'iscrizione e del voto già registrato (`registraVoto`), l'unicità della matricola (`aggiungiStudente`), l'iscrizione doppia (`Appello.iscrivi`). Le violazioni sono eccezioni di una piccola gerarchia con radice `DominioException`: `MatricolaDuplicataException`, `VotoNonValidoException`, `VotoDuplicatoException`, `IscrizioneNonValidaException`, `EntitaNonTrovataException`. La vista cattura la sola radice e mostra il messaggio, così aggiungere una nuova regola non richiede di toccare la grafica.

2.2.4 Proteggere lo stato interno

Problema. Se `Ateneo` restituisse le proprie liste, un chiamante potrebbe svuotarle o alterarle senza passare dalle regole.

Soluzione. I metodi di lettura restituiscono copie non modificabili (`List.copyOf`); gli iscritti di un appello sono esposti come vista non modificabile (`Collections.unmodifiableSet`); le entità sono classi `final` con soli campi `final` e senza setter (`Studente.java`).

Capitolo 3

Sviluppo

3.1 Testing automatizzato

I test sono scritti con JUnit 5 e si lanciano con `gradle test`. Sono 25, in due classi.

- `AteneoTest` (21 test) prova il modello senza grafica: identificativi progressivi; matricola duplicata, che non consuma un identificativo; dati vuoti; corso inesistente; iscrizione doppia; voto tra 18 e 30; lode solo con 30; voto senza iscrizione; voto duplicato; studente o appello inesistenti; media aritmetica; media ponderata (per esempio con 30 su 6 CFU e 24 su 12 CFU la media vale $(30 \cdot 6 + 24 \cdot 12)/18 = 26$); lode contata come 30; medie vuote senza voti; media del corso su più studenti; liste non modificabili; notifica degli osservatori solo dopo una modifica riuscita.
- `UniManagerControllerTest` (4 test) prova la lettura della data, il rifiuto delle date non valide, un flusso completo dall'inserimento alle tre medie e la notifica degli osservatori attraverso il controller.

La vista Swing non ha test automatici: è stata provata avviando l'applicazione e usando tutteleschede(inserimenti,iscrizione, voto,medie,messaggid'errore).
.

3.2 Notedi sviluppo

Funzionalità avanzate del linguaggio e delle librerie

- **Stream e lambda.** Verifica del voto già registrato con (`Ateneo.java`,`righe)` filtrodeivotiperstudenteepercorso(`riga 173`, `riga 181`); somme pesate in `MediaPonderata.java`.
- **Optional.** `OptionalDouble` per le medie che possono non esistere (`CalcoloMedia.java, riga 16`).
- **Interfacce funzionali e riferimenti a metodo.** `AteneoListener` è un'interfaccia funzionale; `MainFrame.java, riga 34` usa il riferimento a metodo `Aggiornabile` .
- **Generici.** Un unico metodo riempie qualunque lista a discesa tipizzata (`UtiliGui.riempi`).
- **Collezioninonmodificabili** , descritte nella sezione 2.2.4.
- **Swing.** `JTabbedPane` per le schede; `JTable` con un `DefaultTableModel` a sola lettura (`TabellaDati.setRighe`); `GridBagLayout` per i form (`Form.aggiungi`); un `ChangeListener` che abilita la casella "30 e lode" solo quando il voto scelto è 30 (`VotiPanel.java`,`righe48-50`).

Codice di terze parti e strumenti

Non è stato copiato codice da altri progetti. Per questa relazione è stata usata la struttura previstadal corso di oop(programmazione ad oggetti)

Capitolo 4

Commenti finali

4.1 Autovalutazione e lavori futuri

Punti di forza. Il modello è separato dalla grafica e fa rispettare le regole del dominio da solo;

aggiungere un nuovo tipo di media non richiede di modificare `Ateneo`.

Limiti. I dati si perdono alla chiusura, perché non c'è persistenza. `Ateneo` concentra la gestione di quattro tipi di entità e potrebbe essere suddivisa in più classi con responsabilità più piccole. Non si possono modificare o cancellare elementi già inseriti. La grafica non ha test automatici.

Appendice A

Guida utente

A.1 Avvio

- `gradle run` avvia l'applicazione;
- `gradle test` esegueitest

All'avvio si apre la finestra con quattro schede. I dati sono vuoti e vengono persi alla chiusura.

A.2 Uso

L'ordine consigliato è: inserire studenti e corsi, creare un appello, iscrivere gli studenti, registrare i voti, consultare le medie. Ogni modifica compare subito in tutte le schede.

1. **Studenti** (Figura A.1). Compilare nome, cognome e matricola e premere "Aggiungi studente". Una matricola già usata viene rifiutata.
2. **Corsi** (Figura A.2). Scrivere il nome, scegliere i CFU (1–30) e premere "Aggiungi corso".
3. **Appelli** (Figura A.3). A sinistra si crea un appello scegliendo il corso, scrivendo la data nel formato `aaaa-mm-gg` e il luogo. A destra si sceglie un appello e uno studente e si preme "Iscrivi studente".
4. **Voti e medie** (Figura A.4). A sinistra si sceglie lo studente, l'appello e il voto (da 18 a 30); la casella "30 e lode" si abilita solo con 30. Lo studente deve essere già iscritto all'appello. A destra si scelgono uno studente o un corso e si premono i pulsanti per vedere le medie.

Quando un'operazione viola una regola (per esempio un voto per uno studente non iscritto) compare una finestra con il motivo e i dati restano invariati.

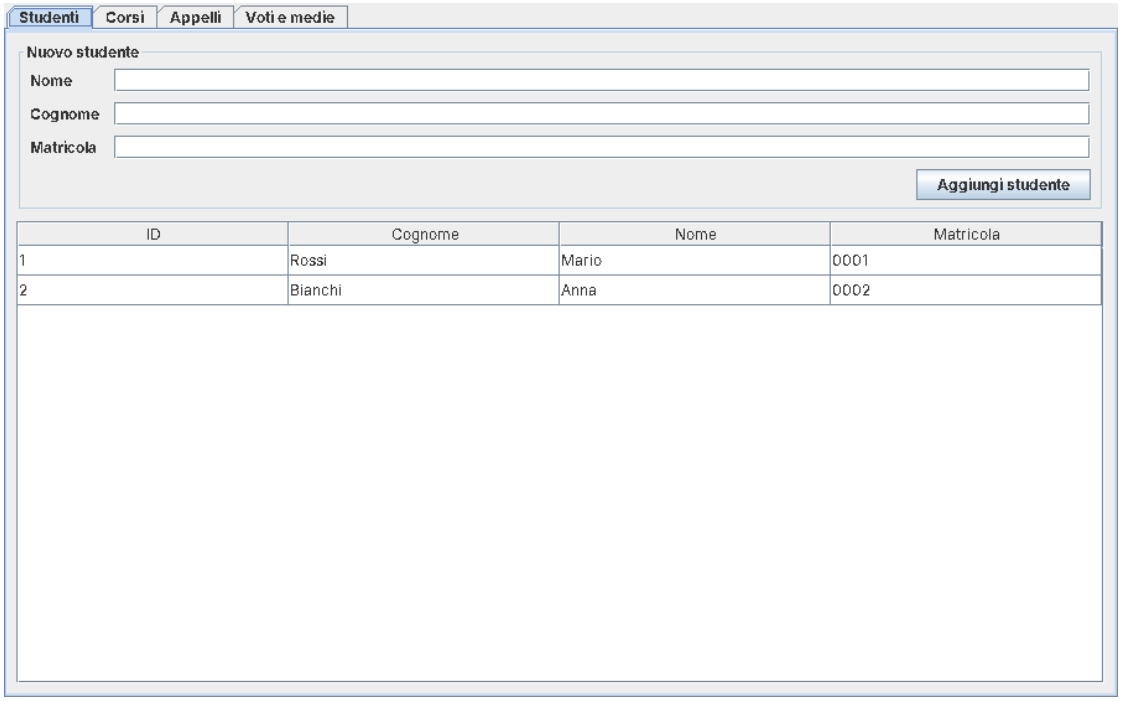


Figura A.1: Scheda Studenti.

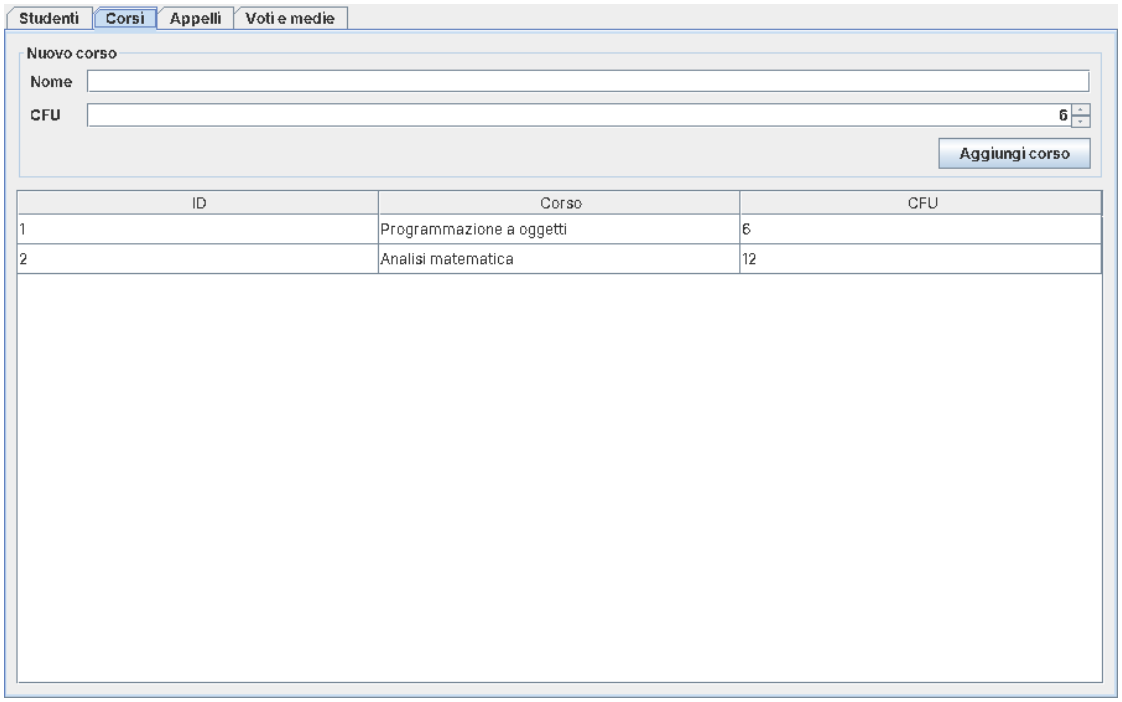


Figura A.2: Scheda Corsi.

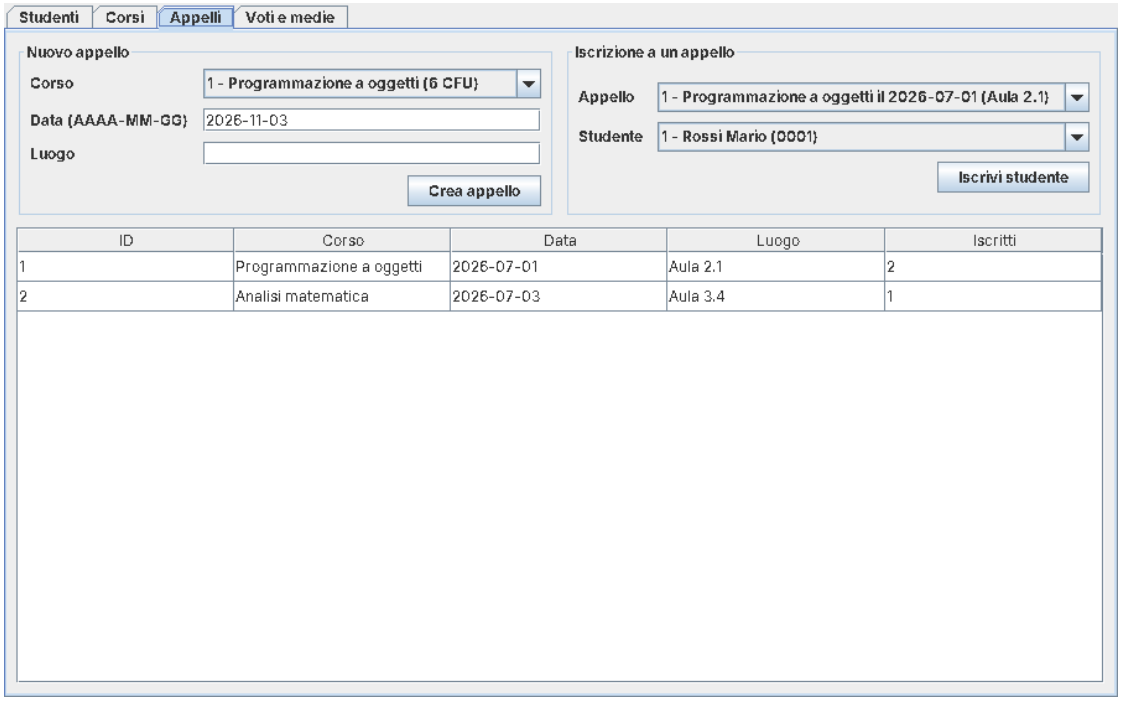


Figura A.3: Scheda Appelli.

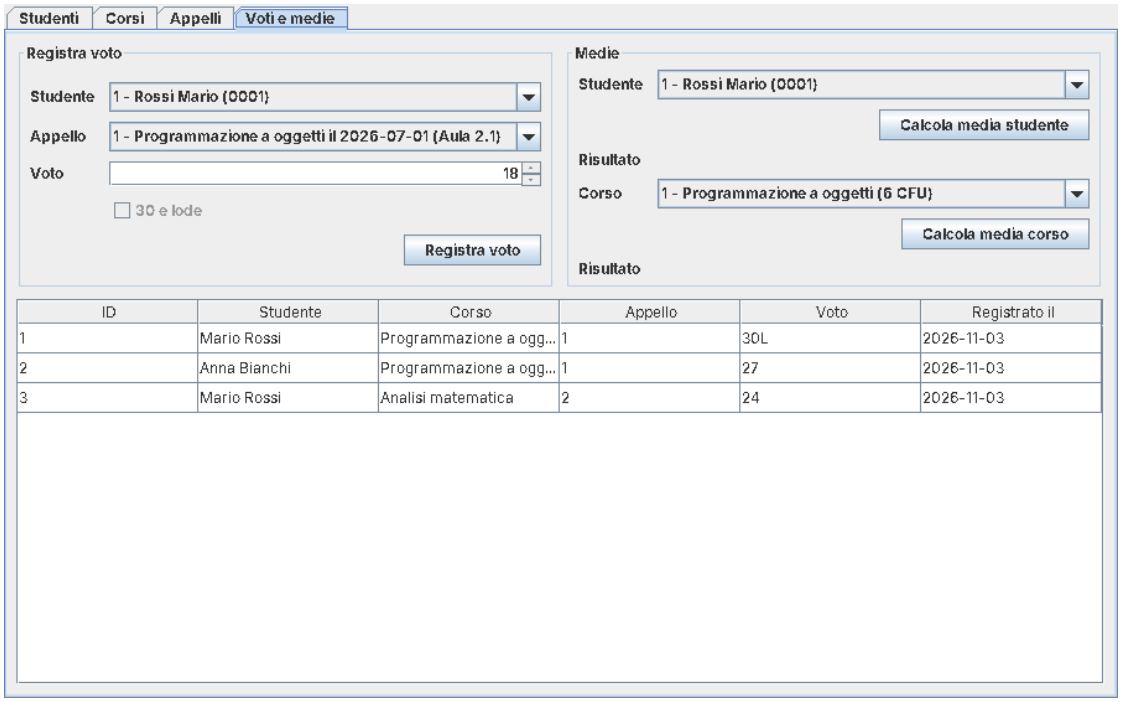


Figura A.4: Scheda Voti e medie.

Appendice B